

目次

本書について	3
改訂履歴	3
読み方	3
1. このサンプルで確認すること	4
1.1 試験対象と業務ルール	4
1.2 シナリオ一覧	4
2. OpenAPI からシナリオへ	5
2.1 ファイルの対応	5
2.2 正常系の流れと値の引き継ぎ	6
2.3 外部 JSON を使う追加シナリオ	7
3. 実施手順	8
3.1 準備	8
3.2 サンプル API を起動する	8
3.3 5 シナリオを実行する	8
3.4 証跡と失敗時の対応	9
3.5 手順書を再生成する	9
3.6 終了	9
4. ORD-001 : 生成された手順・詳細	10
4.1 テストシナリオ	10
4.2 HTTP 呼び出し情報	11
4.3 リクエストボディ	11
4.4 期待値・検証式	12
5. ORD-002 : 生成された手順・詳細	14
5.1 テストシナリオ	14
5.2 HTTP 呼び出し情報	14
5.3 リクエストボディ	15
5.4 期待値・検証式	15
6. ORD-003 : 生成された手順・詳細	17
6.1 テストシナリオ	17
6.2 HTTP 呼び出し情報	18
6.3 リクエストボディ	18
6.4 期待値・検証式	19
7. ORD-004 複数注文で在庫を使い切り、取消後に再注文する	21
7.1 テストシナリオ	21
7.2 HTTP 呼び出し情報	23
7.3 リクエストボディ	23
7.4 期待値・検証式	25
8. ORD-005 数量不正を修正して注文し、取消まで確認する	31

8.1 テストシナリオ	31
8.2 HTTP 呼び出し情報	32
8.3 リクエストボディ	32
8.4 期待値・検証式	33
9. 実施記録	36

本書について

本書は、OpenAPI を起点に作成した注文・在庫連携 API の複数ステップテストと、同じ runbook から生成した手順表を組み合わせたテスト手順書のサンプルである。

原稿は Quarto book 形式とし、前付けと 9 章で構成する。本文の章・節は `chapters/`、自動生成の表は `generated/` に置く。PDF は ddq の様式で発行し、HTML は章ごとのページで確認する。

改訂履歴

日付	内容
2026-09-23	注文・在庫連携の 3 シナリオを作成。ddq による PDF 発行と章・節別 book 構造に対応
2026-09-23	外部 JSON を使う売切後の再注文・入力訂正を追加。全 5 シナリオに拡充

読み方

- 概要とシナリオの作り方を確認する場合は第 1 章・第 2 章を読む。
- 実行・文書再生成の方法は第 3 章、生成された手順・詳細は第 4 章～第 8 章を読む。
- 外部 JSON の読み方は第 2 章、追加シナリオは第 7・8 章を読む。
- 実施記録は第 9 章に残す。生成表の良否欄には実行結果を自動転記しない。

同梱の API はローカル教材であり、実サービスの品質を判定するものではない。

1. このサンプルで確認すること

商品を 2 個注文したときに在庫が 5 個から 3 個へ減り、その注文を取り消すと 5 個へ戻ることを確認する。前の応答から取得した注文 ID を後続のリクエストへ引き継ぎ、注文と在庫の両方の保存状態を検証する。

この文書の「生成された手順・詳細」以下の表は、同梱の実行用 runbook から `runnora-instructions generate` で生成した原稿を差し込んでいる。目的、業務ルール、準備・実行・終了手順、読み方、結果記録欄は人が補う部分である。生成表の良否欄は未記入のままとし、実行レポートとは区別する。

1.1 試験対象と業務ルール

項目	このサンプルの取り決め
対象	同梱の <code>mock/main.go</code> が提供するローカル教材 API
接続先	<code>http://127.0.0.1:18081</code> 。外部システムや DB は使用しない
初期状態	商品 <code>PEN-001</code> 、引当可能在庫 5 個、注文 0 件
注文確定	<code>POST /orders</code> が 201。数量分の在庫を減らし、状態を <code>confirmed</code> にする
注文取消	<code>POST /orders/{orderId}/cancel</code> が 200。在庫を戻し、状態を <code>cancelled</code> にする
在庫不足	409 と <code>INSUFFICIENT_STOCK</code> 。在庫を変更しない
二重取消	409 と <code>ORDER_ALREADY_CANCELLED</code> 。在庫を追加で戻さない
再実行	各 runbook の先頭で初期化する。注文 ID の採番値は初期化しない

ノート

実サービスでは、再キャンセルを 200 として扱う設計もある。本サンプルでは 409 を返す契約とした。仕様の例示であり、全 API に共通するルールではない。

1.2 シナリオ一覧

ID	観点	入力	ステップ数	最終確認
ORD-001	作成・照会・在庫引当・取消・再照会	注文数量 2	9	注文 <code>cancelled</code> 、在庫 5
ORD-002	在庫不足の拒否後の状態	在庫 5 に対して数量 6	4	409、在庫 5 のまま
ORD-003	同一 ID への二重キャンセル	数量 2、同じ注文を 2 回取消	8	2 回目 409、在庫 5 (7 にならない)
ORD-004	売切・一部取消・再注文	外部 JSON、注文 2 個 + 3 個 + 再注文 1 個	20	他注文の状態維持、全件取消後に在庫 5
ORD-005	入力エラーからの回復	外部 JSON、数量 0 → 数量 1	9	400 後に訂正注文が成功、取消後に在庫 5

ステップ数には初期化と `bind` を含む。合計 50 ステップ、HTTP 呼び出し 44 回、ID 保存 6 回。ORD-004・005 は本文と期待レスポンスを外部 JSON へ分けた追加シナリオである。404、並行注文、認証、永続 DB、障害時のトランザクションは今回の 5 シナリオの対象外である。ORD-002 は在庫不変までを検証し、拒否時の注文レコード不存在は検証していない。

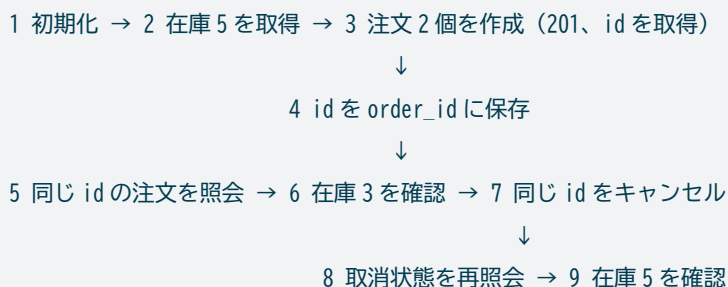
2. OpenAPI からシナリオへ

2.1 ファイルの対応

ファイル／フォルダ（サンプルのルート基準）	役割・確認ポイント
openapi.yaml	4 つの業務 API と初期化 API の入出力・応答契約
baseline/runbooks/generated/	OpenAPI から実際に生成した API 単位の template と suite
baseline/cases/generated/	雛形に対応する入力と期待ステータス
runbooks/order-lifecycle.yml	正常系。API 間の実行順と注文 ID の引き継ぎを記述
runbooks/insufficient-stock.yml	在庫不足の応答と、その後の在庫を確認
runbooks/double-cancel.yml	同じ ID で 2 回取消し、二重加算を検出
runbooks/stock-recovery.yml	2 件で売切、一部取消後に再注文する追加シナリオ
runbooks/invalid-recovery.yml	数量不正を拒否し、訂正して注文・取消する追加シナリオ
fixtures/requests/	追加シナリオが送信する JSON 本文
fixtures/responses/	追加シナリオが比較する期待 JSON 本文（実測値ではない）
docs/generated/	実行用 runbook から生成した手順・HTTP・本文・検証式の表
docs/index.qmd	採番しない前付け
docs/chapters/	章ごとのフォルダと、節ごとの QMD。生成表を include する
docs/_quarto.yml	book の章順・表題・発行設定
docs/design-doc.pdf	ddq で発行した PDF。横向きページ・表の分割・ページ番号を確認する
docs/_book/	章ごとの HTML と共通アセット。配布時はフォルダ一式を渡す
reports/<日時>/	利用者が実行したときのレポート。版管理対象外

baseline の生成物は API ごとの出発点である。どの API をどの順で呼ぶか、初期在庫、注文数量、ID の引き継ぎ、取消前後の整合性は OpenAPI だけから自動で決まらない。本サンプルではそれらを runbooks/ に記述し、desc に日本語の目的を付けている。作成済み注文 ID を必要とする baseline の照会・取消 suite は、そのまま一連の業務テストにはならない。

2.2 正常系の流れと値の引き継ぎ



値	取得元	使用先
order_id	steps.create_order.res.body.id	注文照会と取消の URL、応答 ID の一致確認
初期在庫	steps.stock_before.res.body.available	引当後は初期在庫-2、取消後は初期在庫との一致を検証
状態	各応答の body.status	作成・初回照会では confirmed、取消・再照会では cancelled

実行用 YAML の ID 保存部分は次のようになる。

```

save_order_id:
  desc: create_order の応答 body.id を order_id に保存し、後続 URL へ引き継ぐ
  bind:
    order_id: steps.create_order.res.body.id
get_order:
  desc: 保存した ID で注文を照会する
  req:
    /orders/{{ order_id }}:
      get:
        body: null
  
```

注文 ID を ORD-000001 と固定しない。採番値が実行ごとに変わっても、作成した注文を追跡できる。生成表では実行時の式をそのまま保持する。実測 ID へ展開された値ではない。現行の変換ツールは bind の代入式を独立した詳細表に出さないため、上の対応表と元 YAML で確認する。

2.3 外部 JSON を使う追加シナリオ

ORD-004・005では、`fixtures/requests/` に送信する JSON を、`fixtures/responses/` に期待レスポンスを置く。元の 3 シナリオの YAML 直接記述と見比べられる構成である。

```
vars:
  req_order_1: json://../fixtures/requests/order-1.json
  res_confirmed_1: json://../fixtures/responses/confirmed-1.json
```

`json://` の相対パスは runbook の配置場所が基準になる。本文には `application/json: "{vars.req_order_1}"` を指定する。期待 JSON は実行前に用意した比較データであり、実測レスポンスを保存したものではない。

ORD-004: A を 2 個注文 → B を 3 個注文 → 売切で追加注文 409

- A 取消で在庫 2 → B の確定状態を確認 → C を 1 個再注文
- A の取消状態と C の保存内容を確認
- B と C も取消 → 在庫 5

ORD-005: 数量 0 で 400 → 在庫 5 を維持 → 数量 1 に訂正して 201

- 保存した ID で照会 → 取消 → 在庫 5

注文の期待 JSON には `sku・quantity・status` を記載する。毎回変わる ID は `compare(current.res.body, vars.res_confirmed_1, ["id"])` の比較から除外し、空でないこと、保存した ID との一致、他の注文と異なることを別の検証式で確認する。在庫・初期化・エラーの本文は除外なしで比較する。HTTP ステータスも各ステップで検証する。

手順書のシナリオ表にはファイル名を、本文・期待値の詳細表には JSON の構造と値を載せる。比較対象の除外や条件は検証式を正として確認する。JSON を書き換えた場合は再実行と手順書の再生成が必要である。

3. 実施手順

3.1 準備

Windows/PowerShell、Go、runnora 実行ファイルを使用する。手順書の発行には Quarto と ddq 2.4.0 を使用する。完成 PDF・HTML の閲覧だけならこれらのツールは不要である。PowerShell でサンプルのルート `examples/order-workflow` に移動する。

```
# リポジトリのルートから
Set-Location examples/order-workflow
New-Item -ItemType Directory -Force bin | Out-Null
go build -o bin/order-api.exe ./mock
if ($LASTEXITCODE -ne 0) { throw 'サンプル API のビルド失敗' }
```

18081 番ポートが未使用であることを確認する。API と runbook の接続先は同じ固定ポートである。`config.yaml` の Oracle DSN は空、前後処理 SQL は空配列である。

3.2 サンプル API を起動する

ターミナル A で次を実行し、起動メッセージを確認する。

```
./bin/order-api.exe
```

この API は応答を固定で返すだけでなく、注文・在庫の状態をメモリに保持する。シナリオ先頭の `POST /test/reset` が注文を消去し、在庫を 5 へ戻す。同一プロセスに対するシナリオは順番に実行する。

3.3 5 シナリオを実行する

ターミナル B も同じサンプルフォルダへ移動し、次を実行する。

```
# 既定では隣接する runnora リポジトリの runnora.exe を使う
./scripts/run.ps1

# 配置が異なる場合は実行ファイルを明示する
# ./scripts/run.ps1 -Runnora C:/tools/runnora.exe
```

スクリプトは ORD-001~005 を順番に実行し、いずれかが非 0 終了なら停止する。各シナリオの成功は「期待どおり」を意味する。ORD-002 と ORD-003 の 409 は期待した業務エラーなので、検証式が成立すればテストとしては成功となる。

ORD-004・005 では、外部 JSON の期待本文と実際のレスポンスを比較する。入力エラーの 400 や売切時の 409 も期待どおりなら成功となる。

1 シナリオだけを実行する場合は次のようにする。

```
$runnora = '..\..\..\runnora/runnora.exe'
& $runnora run --config config.yaml runbooks/order-lifecycle.yml
if ($LASTEXITCODE -ne 0) { throw 'ORD-001 失敗' }
```

成功時も HTTP ステータスだけでは合格としない。本文の ID・商品・数量・状態・在庫など、後掲の検証式が成立することを確認する。OpenAPI の全スキーマ制約を自動検証しているわけではない。

3.4 証跡と失敗時の対応

reports/<実行日時>/ の各テキストレポートで失敗 0 件と終了コード 0 を確認する。runnora の集計は runbook 単位のため、「1 成功」は HTTP 呼び出しが 1 回という意味ではない。実施記録欄に日時、実施者、runnora の版、対象ソースの版、証跡の場所を記入する。実測レスポンスを含む詳細調査が必要なら、単独実行に `--trace` を追加してログを保存する。

失敗した場合は、その時のログと終了コードを残し、最初に失敗した検証式と実測値を比較する。シナリオ途中からの再開はせず、原因修正後に先頭から実行して初期状態をそろえる。再実行のレポートは別の日時フォルダに保存する。

3.5 手順書を再生成する

```
./scripts/build-docs.ps1
# 配置が異なる場合:
# ./scripts/build-docs.ps1 `
#   -Runnora C:/tools/runnora.exe `
#   -Ddq C:/tools/quarto-template-2.4.0/ddq.exe
# HTML または PDF のみ発行する場合:
# ./scripts/build-docs.ps1 -Format Html
# ./scripts/build-docs.ps1 -Format Pdf
```

このスクリプトは、OpenAPI から baseline を生成し、実行用 runbook から QMD を生成した後、`ddq pdf docs`、`ddq html docs` の順で PDF と HTML を発行する。API の起動は不要。HTML は `docs/_book/index.html`、PDF は `docs/design-doc.pdf` を開く。HTML の配布は `_book/` 全体を対象とする。全文検索には HTTP サーバからの閲覧が必要である。

原稿だけを修正して再発行する場合は、サンプルのルートで次を実行する。

```
$ddq = '..\..\..\design-doc-quarto-template/cli/target/release/ddq.exe'
& $ddq pdf docs
if ($LASTEXITCODE -ne 0) { throw 'PDF 発行失敗' }
& $ddq html docs
if ($LASTEXITCODE -ne 0) { throw 'HTML 発行失敗' }
```

既定の `ddq` パスは開発環境向けである。配布環境では展開した `ddq.exe` のパスを指定する。`ddq 2.4.0` は形式ごとに `_book/` を作り直すため、両方残す場合は PDF を先に発行する。PDF は `docs/design-doc.pdf` に複写されるので、その後の HTML 発行でも残る。`.template-version` と使用する `ddq` の版・機構が異なる場合、`ddq` は停止する。更新時は `./scripts/build-docs.ps1 -UpdateTemplate` を指定して `ddq update docs` を実行し、機構ファイルの差分を確認する。`lib.typ`、`typst-*.typ`、`_quarto-publish.yml` は `ddq` が配置する発行用ファイルなので手編集しない。

生成された表は直接編集せず、`runbooks/` の説明・リクエスト・検証式を修正して再生成する。OpenAPI を変更した場合は業務 runbook も手動で追従させる。

3.6 終了

ターミナル A で `Ctrl+C` を押して API を終了する。メモリ上の注文・在庫は破棄される。再起動すると在庫 5・注文 0 件・採番初期値から始まる。

4. ORD-001 : 生成された手順・詳細

前提：専用サンプル API が起動している。最初の手順で初期化する。合格条件：注文の内容と ID が一貫し、在庫が 5→3→5、状態が confirmed→cancelled になる。

4.1 テストシナリオ

表 4.1-1 テストシナリオ

シナリオ	ORD-001 注文の作成からキャンセルまで			
手順番号	試験手順	期待結果		結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	サンプル専用 API で在庫 5・注文 0 件に初期化する URL : 表 4.2-1-[1]	表 4.3-1	期待値 : [200] 表 4.4-1	
2	商品 PEN-001 の初期在庫が 5 個であることを確認する URL : 表 4.2-1-[2]		期待値 : [200] 表 4.4-2	
3	商品を 2 個注文し、採番された ID・商品・数量・確定状態を確認する URL : 表 4.2-1-[3]	表 4.3-2	期待値 : [201] 表 4.4-3	
4	create_order の応答 body.id を order_id に保存し、後続 URL へ 引き継ぐ 変数を設定			
5	保存した ID で注文を照会し、作成内容が保持されていることを確認する URL : 表 4.2-1-[5]		期待値 : [200] 表 4.4-4	
6	注文 2 個分の在庫が引き当てられ、5 個から 3 個に減ったことを確認する URL : 表 4.2-1-[6]		期待値 : [200] 表 4.4-5	
7	同じ ID の注文をキャンセルし、取消状態になったことを確認 する URL : 表 4.2-1-[7]	表 4.3-3	期待値 : [200] 表 4.4-6	
8	再照会し、取消状態が保存されていることを確認する URL : 表 4.2-1-[8]		期待値 : [200] 表 4.4-7	
9	キャンセルにより在庫が初期値 5 個に戻ったことを確認する URL : 表 4.2-1-[9]		期待値 : [200] 表 4.4-8	

4.2 HTTP 呼び出し情報

表 4.2-1 HTTP 呼び出し情報（ORD-001 注文の作成からキャンセルまで）

手順	Method	URL	Headers	Query
1	POST	http://127.0.0.1:18081/test/reset		
2	GET	http://127.0.0.1:18081/inventory/PEN-001		
3	POST	http://127.0.0.1:18081/orders		
5	GET	http://127.0.0.1:18081/orders/{ order_id }		
6	GET	http://127.0.0.1:18081/inventory/PEN-001		
7	POST	http://127.0.0.1:18081/orders/{ order_id }/cancel		
8	GET	http://127.0.0.1:18081/orders/{ order_id }		
9	GET	http://127.0.0.1:18081/inventory/PEN-001		

4.3 リクエストボディ

表 4.3-1 リクエストボディ（ORD-001 注文の作成からキャンセルまで／手順 1）

JSON path	値
\$	{ }

表 4.3-2 リクエストボディ（ORD-001 注文の作成からキャンセルまで／手順 3）

JSON path	値
\$.quantity	2
\$.sku	PEN-001

表 4.3-3 リクエストボディ（ORD-001 注文の作成からキャンセルまで／手順 7）

JSON path	値
\$	{ }

4.4 期待値・検証式

表 4.4-1 期待値・検証条件（ORD-001 注文の作成からキャンセルまで／手順 1）

区分	出典	内容
検証式	order-lifecycle.yml:7	current.res.status == 200 && current.res.body.available == 5 && current.res.body.orderCount == 0
ステータス	検証式から抽出	HTTP 200

表 4.4-2 期待値・検証条件（ORD-001 注文の作成からキャンセルまで／手順 2）

区分	出典	内容
検証式	order-lifecycle.yml:18	current.res.status == 200 && current.res.body.sku == "PEN-001" && current.res.body.available == 5
ステータス	検証式から抽出	HTTP 200

表 4.4-3 期待値・検証条件（ORD-001 注文の作成からキャンセルまで／手順 3）

区分	出典	内容
検証式	order-lifecycle.yml:28	current.res.status == 201 && current.res.body.id != "" && current.res.body.sku == "PEN-001" && current.res.body.quantity == 2 && current.res.body.status == "confirmed"
ステータス	検証式から抽出	HTTP 201

表 4.4-4 期待値・検証条件（ORD-001 注文の作成からキャンセルまで／手順 5）

区分	出典	内容
検証式	order-lifecycle.yml:47	current.res.status == 200 && current.res.body.id == order_id && current.res.body.sku == "PEN-001" && current.res.body.quantity == 2 && current.res.body.status == "confirmed"
ステータス	検証式から抽出	HTTP 200

表 4.4-5 期待値・検証条件（ORD-001 注文の作成からキャンセルまで／手順 6）

区分	出典	内容
検証式	order-lifecycle.yml:59	current.res.status == 200 && current.res.body.available == steps.stock_before.res.body.available - 2
ステータス	検証式から抽出	HTTP 200

表 4.4-6 期待値・検証条件（ORD-001 注文の作成からキャンセルまで／手順 7）

区分	出典	内容
検証式	order-lifecycle.yml:68	current.res.status == 200 && current.res.body.id == order_id && current.res.body.sku == "PEN-001" && current.res.body.quantity == 2 && current.res.body.status == "cancelled"
ステータス	検証式から抽出	HTTP 200

表 4.4-7 期待値・検証条件（ORD-001 注文の作成からキャンセルまで／手順 8）

区分	出典	内容
検証式	order-lifecycle.yml:81	current.res.status == 200 && current.res.body.id == order_id && current.res.body.sku == "PEN-001" && current.res.body.quantity == 2 && current.res.body.status == "cancelled"
ステータス	検証式から抽出	HTTP 200

表 4.4-8 期待値・検証条件（ORD-001 注文の作成からキャンセルまで／手順 9）

区分	出典	内容
検証式	order-lifecycle.yml:93	current.res.status == 200 && current.res.body.available == steps.stock_before.res.body.available
ステータス	検証式から抽出	HTTP 200

5. ORD-002：生成された手順・詳細

前提：初期化により在庫 5。合格条件：数量 6 の注文が 409 となり、再照会でも在庫 5 を維持する。最後の再照会を省くと「エラーを返したのに在庫だけ減った」不具合を見逃す。

5.1 テストシナリオ

表 5.1-1 テストシナリオ

シナリオ	ORD-002 在庫不足時に在庫を変更しない			
手順番号	試験手順	期待結果		結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	サンプル専用 API で在庫 5・注文 0 件に初期化する URL：表 5.2-1-[1]	表 5.3-1	期待値：[200] 表 5.4-1	
2	初期在庫が 5 個であることを確認する URL：表 5.2-1-[2]		期待値：[200] 表 5.4-2	
3	在庫を超える 6 個を注文し、在庫不足エラーを確認する URL：表 5.2-1-[3]	表 5.3-2	期待値：[409] 表 5.4-3	
4	失敗した注文によって在庫が減っていないことを再照会して 確認する URL：表 5.2-1-[4]		期待値：[200] 表 5.4-4	

5.2 HTTP 呼び出し情報

表 5.2-1 HTTP 呼び出し情報（ORD-002 在庫不足時に在庫を変更しない）

手順	Method	URL	Headers	Query
1	POST	http://127.0.0.1:18081/test/reset		
2	GET	http://127.0.0.1:18081/inventory/PEN-001		
3	POST	http://127.0.0.1:18081/orders		
4	GET	http://127.0.0.1:18081/inventory/PEN-001		

5.3 リクエストボディ

表 5.3-1 リクエストボディ（ORD-002 在庫不足時に在庫を変更しない／手順 1）

JSON path	値
\$	{}

表 5.3-2 リクエストボディ（ORD-002 在庫不足時に在庫を変更しない／手順 3）

JSON path	値
\$.quantity	6
\$.sku	PEN-001

5.4 期待値・検証式

表 5.4-1 期待値・検証条件（ORD-002 在庫不足時に在庫を変更しない／手順 1）

区分	出典	内容
検証式	insufficient-stock.yml:7	current.res.status == 200 && current.res.body.available == 5 && current.res.body.orderCount == 0
ステータス	検証式から抽出	HTTP 200

表 5.4-2 期待値・検証条件（ORD-002 在庫不足時に在庫を変更しない／手順 2）

区分	出典	内容
検証式	insufficient-stock .yml:18	current.res.status == 200 && current.res.body.sku == "PEN-001" && current.res.body.available == 5
ステータス	検証式から抽出	HTTP 200

表 5.4-3 期待値・検証条件（ORD-002 在庫不足時に在庫を変更しない／手順 3）

区分	出典	内容
検証式	insufficient-stock .yml:28	current.res.status == 409 && current.res.body.code == "INSUFFICIENT_STOCK"
ステータス	検証式から抽出	HTTP 409

表 5.4-4 期待値・検証条件（ORD-002 在庫不足時に在庫を変更しない／手順 4）

区分	出典	内容
検証式	insufficient-stock .yaml:40	current.res.status == 200 && current.res.body.available == steps.stock_before.res.body.available
ステータス	検証式から抽出	HTTP 200

6. ORD-003 : 生成された手順・詳細

前提 : 初期化により在庫 5。合格条件 : 1 回目の取消は 200、同じ ID への 2 回目は 409。注文は cancelled を維持し、最終在庫は 5 である。誤って 2 回とも加算すると 7 となり、最終検証が失敗する。

6.1 テストシナリオ

表 6.1-1 テストシナリオ

シナリオ	ORD-003 二重キャンセルで在庫を二重加算しない			
手順番号	試験手順	期待結果		結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	サンプル専用 API で在庫 5・注文 0 件に初期化する URL : 表 6.2-1-[1]	表 6.3-1	期待値 : [200] 表 6.4-1	
2	初期在庫が 5 個であることを確認する URL : 表 6.2-1-[2]		期待値 : [200] 表 6.4-2	
3	商品を 2 個注文し、確定状態を確認する URL : 表 6.2-1-[3]	表 6.3-2	期待値 : [201] 表 6.4-3	
4	create_order の応答 body.id を order_id に保存し、2 回とも同じ注文を取り消す 変数を設定			
5	1 回目のキャンセルが成功することを確認する URL : 表 6.2-1-[5]	表 6.3-3	期待値 : [200] 表 6.4-4	
6	同じ注文を再キャンセルし、取消済みエラーを確認する URL : 表 6.2-1-[6]	表 6.3-4	期待値 : [409] 表 6.4-5	
7	エラー後も注文が取消状態のままであることを確認する URL : 表 6.2-1-[7]		期待値 : [200] 表 6.4-6	
8	在庫が 5 個に戻り、二重加算された 7 個になっていないことを確認する URL : 表 6.2-1-[8]		期待値 : [200] 表 6.4-7	

6.2 HTTP 呼び出し情報

表 6.2-1 HTTP 呼び出し情報 (ORD-003 二重キャンセルで在庫を二重加算しない)

手順	Method	URL	Headers	Query
1	POST	http://127.0.0.1:18081/test/reset		
2	GET	http://127.0.0.1:18081/inventory/PEN-001		
3	POST	http://127.0.0.1:18081/orders		
5	POST	http://127.0.0.1:18081/orders/{ order_id }/cancel		
6	POST	http://127.0.0.1:18081/orders/{ order_id }/cancel		
7	GET	http://127.0.0.1:18081/orders/{ order_id }		
8	GET	http://127.0.0.1:18081/inventory/PEN-001		

6.3 リクエストボディ

表 6.3-1 リクエストボディ (ORD-003 二重キャンセルで在庫を二重加算しない／手順 1)

JSON path	値
\$	{}

表 6.3-2 リクエストボディ (ORD-003 二重キャンセルで在庫を二重加算しない／手順 3)

JSON path	値
\$.quantity	2
\$.sku	PEN-001

表 6.3-3 リクエストボディ (ORD-003 二重キャンセルで在庫を二重加算しない／手順 5)

JSON path	値
\$	{}

表 6.3-4 リクエストボディ (ORD-003 二重キャンセルで在庫を二重加算しない／手順 6)

JSON path	値
\$	{}

6.4 期待値・検証式

表 6.4-1 期待値・検証条件（ORD-003 二重キャンセルで在庫を二重加算しない／手順 1）

区分	出典	内容
検証式	double-cancel.yml:7	current.res.status == 200 && current.res.body.available == 5 && current.res.body.orderCount == 0
ステータス	検証式から抽出	HTTP 200

表 6.4-2 期待値・検証条件（ORD-003 二重キャンセルで在庫を二重加算しない／手順 2）

区分	出典	内容
検証式	double-cancel.yml:18	current.res.status == 200 && current.res.body.sku == "PEN-001" && current.res.body.available == 5
ステータス	検証式から抽出	HTTP 200

表 6.4-3 期待値・検証条件（ORD-003 二重キャンセルで在庫を二重加算しない／手順 3）

区分	出典	内容
検証式	double-cancel.yml:28	current.res.status == 201 && current.res.body.id != "" && current.res.body.sku == "PEN-001" && current.res.body.quantity == 2 && current.res.body.status == "confirmed"
ステータス	検証式から抽出	HTTP 201

表 6.4-4 期待値・検証条件（ORD-003 二重キャンセルで在庫を二重加算しない／手順 5）

区分	出典	内容
検証式	double-cancel.yml:47	current.res.status == 200 && current.res.body.id == order_id && current.res.body.sku == "PEN-001" && current.res.body.quantity == 2 && current.res.body.status == "cancelled"
ステータス	検証式から抽出	HTTP 200

表 6.4-5 期待値・検証条件（ORD-003 二重キャンセルで在庫を二重加算しない／手順 6）

区分	出典	内容
検証式	double-cancel.yml:60	current.res.status == 409 && current.res.body.code == "ORDER_ALREADY_CANCELLED"
ステータス	検証式から抽出	HTTP 409

表 6.4-6 期待値・検証条件（ORD-003 二重キャンセルで在庫を二重加算しない／手順 7）

区分	出典	内容
検証式	double-cancel.yml:70	current.res.status == 200 && current.res.body.id == order_id && current.res.body.sku == "PEN-001" && current.res.body.quantity == 2 && current.res.body.status == "cancelled"
ステータス	検証式から抽出	HTTP 200

表 6.4-7 期待値・検証条件（ORD-003 二重キャンセルで在庫を二重加算しない／手順 8）

区分	出典	内容
検証式	double-cancel.yml:82	current.res.status == 200 && current.res.body.available == steps.stock_before.res.body.available
ステータス	検証式から抽出	HTTP 200

7. ORD-004 複数注文で在庫を使い切り、取消後に再注文する

外部 JSON を入力・期待本文として使用する。注文 A・B・C の ID を別々に保存し、在庫 5→3→0→2→1→5 と、注文同士の状態の独立性を確認する。

期待 JSON の注文本文には採番 ID を含めない。`compare(..., [".id"])` で ID 以外を照合し、ID の存在・一致・注文間の違いは検証式で別に確認する。

7.1 テストシナリオ

表 7.1-1 テストシナリオ (1/2)

シナリオ	ORD-004 複数注文で在庫を使い切り、取消後に再注文する			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	外部 JSON を使って在庫 5・注文 0 件に初期化する URL : 表 7.2-1-[1]	表 7.3-1 empty.json	期待値 : [200] 表 7.4-1 reset.json	
2	注文 A を 2 個作り、期待 JSON と ID 以外の本文を照合する URL : 表 7.2-1-[2]	表 7.3-2 order-2.json	期待値 : [201] 表 7.4-2 confirmed-2.json	
3	応答 ID を order_a に保存する 変数を設定			
4	注文 A の引当後に在庫 3 を確認する URL : 表 7.2-1-[4]		期待値 : [200] 表 7.4-3 stock-3.json	
5	注文 B を 3 個作り、注文 A と異なる ID を確認する URL : 表 7.2-1-[5]	表 7.3-3 order-3.json	期待値 : [201] 表 7.4-4 confirmed-3.json	
6	応答 ID を order_b に保存する 変数を設定			
7	2 件の注文で在庫を使い切ったことを確認する URL : 表 7.2-1-[7]		期待値 : [200] 表 7.4-5 stock-0.json	
8	売切中の追加注文 1 個を 409 で拒否する URL : 表 7.2-1-[8]	表 7.3-4 order-1.json	期待値 : [409] 表 7.4-6 insufficient.json	
9	追加注文の拒否後も在庫 0 を維持する URL : 表 7.2-1-[9]		期待値 : [200] 表 7.4-7 stock-0.json	

表 7.1-1 テストシナリオ (2/2)

シナリオ	ORD-004 複数注文で在庫を使い切り、取消後に再注文する			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
10	注文 A だけを取り消す URL : 表 7.2-1-[10]	表 7.3-5 empty.json	期待値 : [200] 表 7.4-8 cancelled-2.json	
11	注文 A の 2 個分だけ在庫が戻ることを確認する URL : 表 7.2-1-[11]		期待値 : [200] 表 7.4-9 stock-2.json	
12	注文 A の取消後も注文 B は確定状態を維持する URL : 表 7.2-1-[12]		期待値 : [200] 表 7.4-10 confirmed-3.json	
13	拒否された数量 1 を再注文し、新しい ID で成功する URL : 表 7.2-1-[13]	表 7.3-6 order-1.json	期待値 : [201] 表 7.4-11 confirmed-1.json	
14	応答 ID を order_c に保存する 変数を設定			
15	再注文の引当後に在庫 1 を確認する URL : 表 7.2-1-[15]		期待値 : [200] 表 7.4-12 stock-1.json	
16	取消済みの注文 A を再照会する URL : 表 7.2-1-[16]		期待値 : [200] 表 7.4-13 cancelled-2.json	
17	再注文 C の保存内容を確認する URL : 表 7.2-1-[17]		期待値 : [200] 表 7.4-14 confirmed-1.json	
18	注文 B を取り消し、3 個分を戻す URL : 表 7.2-1-[18]	表 7.3-7 empty.json	期待値 : [200] 表 7.4-15 cancelled-3.json	
19	再注文 C も取り消し、1 個分を戻す URL : 表 7.2-1-[19]	表 7.3-8 empty.json	期待値 : [200] 表 7.4-16 cancelled-1.json	
20	全注文の取消後に在庫 5 へ復元したことを確認する URL : 表 7.2-1-[20]		期待値 : [200] 表 7.4-17 stock-5.json	

7.2 HTTP 呼び出し情報

表 7.2-1 HTTP 呼び出し情報（ORD-004 複数注文で在庫を使い切り、取消後に再注文する）

手順	Method	URL	Headers	Query
1	POST	http://127.0.0.1:18081/test/reset		
2	POST	http://127.0.0.1:18081/orders		
4	GET	http://127.0.0.1:18081/inventory/PEN-001		
5	POST	http://127.0.0.1:18081/orders		
7	GET	http://127.0.0.1:18081/inventory/PEN-001		
8	POST	http://127.0.0.1:18081/orders		
9	GET	http://127.0.0.1:18081/inventory/PEN-001		
10	POST	http://127.0.0.1:18081/orders/{{ order_a }}/cancel		
11	GET	http://127.0.0.1:18081/inventory/PEN-001		
12	GET	http://127.0.0.1:18081/orders/{{ order_b }}		
13	POST	http://127.0.0.1:18081/orders		
15	GET	http://127.0.0.1:18081/inventory/PEN-001		
16	GET	http://127.0.0.1:18081/orders/{{ order_a }}		
17	GET	http://127.0.0.1:18081/orders/{{ order_c }}		
18	POST	http://127.0.0.1:18081/orders/{{ order_b }}/cancel		
19	POST	http://127.0.0.1:18081/orders/{{ order_c }}/cancel		
20	GET	http://127.0.0.1:18081/inventory/PEN-001		

7.3 リクエストボディ

表 7.3-1 リクエストボディ（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 1）

出典	JSON path	値
参照式	\$	"{{ vars.req_empty }}"
empty.json	\$	{}

表 7.3-2 リクエストボディ（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 2）

出典	JSON path	値
参照式	\$	"{{ vars.req_order_2 }}"
order-2.json	\$.quantity	2
	\$.sku	PEN-001

表 7.3-3 リクエストボディ（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 5）

出典	JSON path	値
参照式	\$	"{{ vars.req_order_3 }}"
order-3.json	\$.quantity	3
	\$.sku	PEN-001

表 7.3-4 リクエストボディ（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 8）

出典	JSON path	値
参照式	\$	"{{ vars.req_order_1 }}"
order-1.json	\$.quantity	1
	\$.sku	PEN-001

表 7.3-5 リクエストボディ（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 10）

出典	JSON path	値
参照式	\$	"{{ vars.req_empty }}"
empty.json	\$	{}

表 7.3-6 リクエストボディ（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 13）

出典	JSON path	値
参照式	\$	"{{ vars.req_order_1 }}"
order-1.json	\$.quantity	1
	\$.sku	PEN-001

表 7.3-7 リクエストボディ (ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 18)

出典	JSON path	値
参照式	\$	"{{ vars.req_empty }}"
empty.json	\$	{}

表 7.3-8 リクエストボディ (ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 19)

出典	JSON path	値
参照式	\$	"{{ vars.req_empty }}"
empty.json	\$	{}

7.4 期待値・検証式

表 7.4-1 期待値・検証条件 (ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 1)

区分	出典	内容
検証式	stock-recovery.yml:25	current.res.status == 200 && compare(current.res.body, vars.res_reset)
ステータス	検証式から抽出	HTTP 200
参照 JSON	reset.json	\$.available: 5
		\$.orderCount: 0
		\$.sku: PEN-001

表 7.4-2 期待値・検証条件 (ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 2)

区分	出典	内容
検証式	stock-recovery.yml:35	current.res.status == 201 && current.res.body.id != "" && compare(current.res.body, vars.res_confirmed_2, [".id"])
ステータス	検証式から抽出	HTTP 201
参照 JSON	confirmed-2.json	\$.quantity: 2
		\$.sku: PEN-001
		\$.status: confirmed

表 7.4-3 期待値・検証条件（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 4）

区分	出典	内容
検証式	stock-recovery.yml:50	current.res.status == 200 && compare(current.res.body, vars.res_stock_3)
ステータス	検証式から抽出	HTTP 200
参照 JSON	stock-3.json	\$.available: 3
		\$.sku: PEN-001

表 7.4-4 期待値・検証条件（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 5）

区分	出典	内容
検証式	stock-recovery.yml:59	current.res.status == 201 && current.res.body.id != "" && current.res.body.id != order_a && compare(current.res.body, vars.res_confirmed_3, ["id"])
ステータス	検証式から抽出	HTTP 201
参照 JSON	confirmed-3.json	\$.quantity: 3
		\$.sku: PEN-001
		\$.status: confirmed

表 7.4-5 期待値・検証条件（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 7）

区分	出典	内容
検証式	stock-recovery.yml:74	current.res.status == 200 && compare(current.res.body, vars.res_stock_0)
ステータス	検証式から抽出	HTTP 200
参照 JSON	stock-0.json	\$.available: 0
		\$.sku: PEN-001

表 7.4-6 期待値・検証条件（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 8）（1／2）

区分	出典	内容
検証式	stock-recovery.yml:83	current.res.status == 409 && compare(current.res.body, vars.res_insufficient)
ステータス	検証式から抽出	HTTP 409

表 7.4-6 期待値・検証条件（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 8）（2／2）

区分	出典	内容
参照 JSON	insufficient.json	\$.code: INSUFFICIENT_STOCK

表 7.4-7 期待値・検証条件（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 9）

区分	出典	内容
検証式	stock-recovery.yml:93	current.res.status == 200 && compare(current.res.body, vars.res_stock_0)
ステータス	検証式から抽出	HTTP 200
参照 JSON	stock-0.json	\$.available: 0
		\$.sku: PEN-001

表 7.4-8 期待値・検証条件（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 10）

区分	出典	内容
検証式	stock-recovery.yml:102	current.res.status == 200 && current.res.body.id == order_a && compare(current.res.body, vars.res_cancelled_2, [".id"])
ステータス	検証式から抽出	HTTP 200
参照 JSON	cancelled-2.json	\$.quantity: 2
		\$.sku: PEN-001
		\$.status: cancelled

表 7.4-9 期待値・検証条件（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 11）

区分	出典	内容
検証式	stock-recovery.yml:113	current.res.status == 200 && compare(current.res.body, vars.res_stock_2)
ステータス	検証式から抽出	HTTP 200
参照 JSON	stock-2.json	\$.available: 2
		\$.sku: PEN-001

表 7.4-10 期待値・検証条件 (ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 12)

区分	出典	内容
検証式	stock-recovery.yml:122	current.res.status == 200 && current.res.body.id == order_b && compare(current.res.body, vars.res_confirmed_3, [".id"])
ステータス	検証式から抽出	HTTP 200
参照 JSON	confirmed-3.json	\$.quantity: 3
		\$.sku: PEN-001
		\$.status: confirmed

表 7.4-11 期待値・検証条件 (ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 13)

区分	出典	内容
検証式	stock-recovery.yml:132	current.res.status == 201 && current.res.body.id != "" && current.res.body.id != order_a && current.res.body.id != order_b && compare(current.res.body, vars.res_confirmed_1, [".id"])
ステータス	検証式から抽出	HTTP 201
参照 JSON	confirmed-1.json	\$.quantity: 1
		\$.sku: PEN-001
		\$.status: confirmed

表 7.4-12 期待値・検証条件 (ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 15)

区分	出典	内容
検証式	stock-recovery.yml:147	current.res.status == 200 && compare(current.res.body, vars.res_stock_1)
ステータス	検証式から抽出	HTTP 200
参照 JSON	stock-1.json	\$.available: 1
		\$.sku: PEN-001

表 7.4-13 期待値・検証条件（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 16）

区分	出典	内容
検証式	stock-recovery.yml:156	current.res.status == 200 && current.res.body.id == order_a && compare(current.res.body, vars.res_cancelled_2, [".id"])
ステータス	検証式から抽出	HTTP 200
参照 JSON	cancelled-2.json	\$.quantity: 2
		\$.sku: PEN-001
		\$.status: cancelled

表 7.4-14 期待値・検証条件（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 17）

区分	出典	内容
検証式	stock-recovery.yml:166	current.res.status == 200 && current.res.body.id == order_c && compare(current.res.body, vars.res_confirmed_1, [".id"])
ステータス	検証式から抽出	HTTP 200
参照 JSON	confirmed-1.json	\$.quantity: 1
		\$.sku: PEN-001
		\$.status: confirmed

表 7.4-15 期待値・検証条件（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 18）

区分	出典	内容
検証式	stock-recovery.yml:176	current.res.status == 200 && current.res.body.id == order_b && compare(current.res.body, vars.res_cancelled_3, [".id"])
ステータス	検証式から抽出	HTTP 200
参照 JSON	cancelled-3.json	\$.quantity: 3
		\$.sku: PEN-001
		\$.status: cancelled

表 7.4-16 期待値・検証条件（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 19）

区分	出典	内容
検証式	stock-recovery.yml:187	current.res.status == 200 && current.res.body.id == order_c && compare(current.res.body, vars.res_cancelled_1, [".id"])
ステータス	検証式から抽出	HTTP 200
参照 JSON	cancelled-1.json	\$.quantity: 1
		\$.sku: PEN-001
		\$.status: cancelled

表 7.4-17 期待値・検証条件（ORD-004 複数注文で在庫を使い切り、取消後に再注文する／手順 20）

区分	出典	内容
検証式	stock-recovery.yml:198	current.res.status == 200 && compare(current.res.body, vars.res_stock_5)
ステータス	検証式から抽出	HTTP 200
参照 JSON	stock-5.json	\$.available: 5
		\$.sku: PEN-001

8. ORD-005 数量不正を修正して注文し、取消まで確認する

数量 0 を 400 で拒否した後、数量 1 で注文をやり直す。在庫 5→4→5 と、訂正後の注文の保存・取消を確認する。

期待 JSON の注文本文には採番 ID を含めない。`compare(..., [".id"])` で ID 以外を照合し、ID の存在・一致・注文間の違いは検証式で別に確認する。

8.1 テストシナリオ

表 8.1-1 テストシナリオ

シナリオ	ORD-005 数量不正を修正して注文し、取消まで確認する			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	外部 JSON を使って在庫 5・注文 0 件に初期化する URL : 表 8.2-1-[1]	表 8.3-1 empty.json	期待値 : [200] 表 8.4-1 reset.json	
2	数量 0 の外部 JSON を送り、400 とエラー本文を確認する URL : 表 8.2-1-[2]	表 8.3-2 order-0.json	期待値 : [400] 表 8.4-2 invalid.json	
3	不正な注文を拒否した後も在庫 5 を維持する URL : 表 8.2-1-[3]		期待値 : [200] 表 8.4-3 stock-5.json	
4	数量 1 の外部 JSON に切り替えて注文する URL : 表 8.2-1-[4]	表 8.3-3 order-1.json	期待値 : [201] 表 8.4-4 confirmed-1.json	
5	応答 ID を order_id に保存する 変数を設定			
6	訂正後の注文で在庫が 4 へ減ることを確認する URL : 表 8.2-1-[6]		期待値 : [200] 表 8.4-5 stock-4.json	
7	訂正後の注文を ID で照会し、保存内容を照合する URL : 表 8.2-1-[7]		期待値 : [200] 表 8.4-6 confirmed-1.json	
8	訂正後の注文を取り消す URL : 表 8.2-1-[8]	表 8.3-4 empty.json	期待値 : [200] 表 8.4-7 cancelled-1.json	
9	取消後に在庫 5 へ戻ったことを確認する URL : 表 8.2-1-[9]		期待値 : [200] 表 8.4-8 stock-5.json	

8.2 HTTP 呼び出し情報

表 8.2-1 HTTP 呼び出し情報（ORD-005 数量不正を修正して注文し、取消まで確認する）

手順	Method	URL	Headers	Query
1	POST	http://127.0.0.1:18081/test/reset		
2	POST	http://127.0.0.1:18081/orders		
3	GET	http://127.0.0.1:18081/inventory/PEN-001		
4	POST	http://127.0.0.1:18081/orders		
6	GET	http://127.0.0.1:18081/inventory/PEN-001		
7	GET	http://127.0.0.1:18081/orders/{{ order_id }}		
8	POST	http://127.0.0.1:18081/orders/{{ order_id }}/cancel		
9	GET	http://127.0.0.1:18081/inventory/PEN-001		

8.3 リクエストボディ

表 8.3-1 リクエストボディ（ORD-005 数量不正を修正して注文し、取消まで確認する／手順 1）

出典	JSON path	値
参照式	\$	"{{ vars.req_empty }}"
empty.json	\$	{}

表 8.3-2 リクエストボディ（ORD-005 数量不正を修正して注文し、取消まで確認する／手順 2）

出典	JSON path	値
参照式	\$	"{{ vars.req_order_0 }}"
order-0.json	\$.quantity	0
	\$.sku	PEN-001

表 8.3-3 リクエストボディ（ORD-005 数量不正を修正して注文し、取消まで確認する／手順 4）（1／2）

出典	JSON path	値
参照式	\$	"{{ vars.req_order_1 }}"
order-1.json	\$.quantity	1

表 8.3-3 リクエストボディ（ORD-005 数量不正を修正して注文し、取消まで確認する／手順 4）（2／2）

出典	JSON path	値
	\$.sku	PEN-001

表 8.3-4 リクエストボディ（ORD-005 数量不正を修正して注文し、取消まで確認する／手順 8）

出典	JSON path	値
参照式	\$	"{{ vars.req_empty }}"
empty.json	\$	{}

8.4 期待値・検証式

表 8.4-1 期待値・検証条件（ORD-005 数量不正を修正して注文し、取消まで確認する／手順 1）

区分	出典	内容
検証式	invalid-recovery.yml:17	current.res.status == 200 && compare(current.res.body, vars.res_reset)
ステータス	検証式から抽出	HTTP 200
参照 JSON	reset.json	\$.available: 5
		\$.orderCount: 0
		\$.sku: PEN-001

表 8.4-2 期待値・検証条件（ORD-005 数量不正を修正して注文し、取消まで確認する／手順 2）

区分	出典	内容
検証式	invalid-recovery.yml:27	current.res.status == 400 && compare(current.res.body, vars.res_invalid)
ステータス	検証式から抽出	HTTP 400
参照 JSON	invalid.json	\$.code: INVALID_REQUEST

表 8.4-3 期待値・検証条件（ORD-005 数量不正を修正して注文し、取消まで確認する／手順 3）（1／2）

区分	出典	内容
検証式	invalid-recovery.yml:37	current.res.status == 200 && compare(current.res.body, vars.res_stock_5)
ステータス	検証式から抽出	HTTP 200

表 8.4-3 期待値・検証条件 (ORD-005 数量不正を修正して注文し、取消まで確認する／手順 3) (2/2)

区分	出典	内容
参照 JSON	stock-5.json	\$.available: 5
		\$.sku: PEN-001

表 8.4-4 期待値・検証条件 (ORD-005 数量不正を修正して注文し、取消まで確認する／手順 4)

区分	出典	内容
検証式	invalid-recovery.yml:46	current.res.status == 201 && current.res.body.id != "" && compare(current.res.body, vars.res_confirmed_1, [".id"])
ステータス	検証式から抽出	HTTP 201
参照 JSON	confirmed-1.json	\$.quantity: 1
		\$.sku: PEN-001
		\$.status: confirmed

表 8.4-5 期待値・検証条件 (ORD-005 数量不正を修正して注文し、取消まで確認する／手順 6)

区分	出典	内容
検証式	invalid-recovery.yml:61	current.res.status == 200 && compare(current.res.body, vars.res_stock_4)
ステータス	検証式から抽出	HTTP 200
参照 JSON	stock-4.json	\$.available: 4
		\$.sku: PEN-001

表 8.4-6 期待値・検証条件 (ORD-005 数量不正を修正して注文し、取消まで確認する／手順 7) (1/2)

区分	出典	内容
検証式	invalid-recovery.yml:70	current.res.status == 200 && current.res.body.id == order_id && compare(current.res.body, vars.res_confirmed_1, [".id"])
ステータス	検証式から抽出	HTTP 200
参照 JSON	confirmed-1.json	\$.quantity: 1

表 8.4-6 期待値・検証条件（ORD-005 数量不正を修正して注文し、取消まで確認する／手順 7）（2／2）

区分	出典	内容
		\$.sku: PEN-001
		\$.status: confirmed

表 8.4-7 期待値・検証条件（ORD-005 数量不正を修正して注文し、取消まで確認する／手順 8）

区分	出典	内容
検証式	invalid-recovery.yml:80	current.res.status == 200 && current.res.body.id == order_id && compare(current.res.body, vars.res_cancelled_1, [".id"])
ステータス	検証式から抽出	HTTP 200
参照 JSON	cancelled-1.json	\$.quantity: 1
		\$.sku: PEN-001
		\$.status: cancelled

表 8.4-8 期待値・検証条件（ORD-005 数量不正を修正して注文し、取消まで確認する／手順 9）

区分	出典	内容
検証式	invalid-recovery.yml:91	current.res.status == 200 && compare(current.res.body, vars.res_stock_5)
ステータス	検証式から抽出	HTTP 200
参照 JSON	stock-5.json	\$.available: 5
		\$.sku: PEN-001

9. 実施記録

次は利用者が自分の実行について記入する欄。サンプル作成時の動作確認結果は [verification.md](#) に別記する。

ID	実施日時・実施者	対象ソース・ツール版	良否	証跡・不具合番号
ORD-001	未記入	未記入	未実施	未記入
ORD-002	未記入	未記入	未実施	未記入
ORD-003	未記入	未記入	未実施	未記入
ORD-004	未記入	未記入	未実施	未記入
ORD-005	未記入	未記入	未実施	未記入

本書の生成は実 API の品質を判定しない。同梱 API による確認は、教材のシナリオ実行・状態変化・文書化が動作することの確認である。